

Санкт-Петербургский государственный электротехнический
университет "ЛЭТИ" им. В.И. Ульянова (Ленина)

Система управления пространствами Rocket.Chat

Автоматизация рутинных задач администрирования



Заказчик:

Иванов Д.В.

Исполнители:

Бесогонов М., Жихарев В., Пикалов И.,
Деменев К., Птицын Д., Тупахин Д., Правдин А.

Основная цель и задачи проекта

Цель:

Автоматизация администрирования Rocket.Chat через веб-интерфейс.

Задачи:

- Массовое управление пользователями (добавление, пароли, экспорт).
- Создание комнат/групп с гибкими настройками.
- Мониторинг активности и интеграция SMTP-рассылок.
- Поддержка нескольких пространств Rocket.Chat.

Используемые технологии / подробности проекта в бэкенд части проекта

Стек технологий:

- Python + FastAPI: Бэкенд на FastAPI, с Pydantic для валидации
- MongoDB: Документо-ориентированная БД
- Docker: Контейнеризация (Dockerfile, .dockerignore)
- Rocket.Chat API: Интеграция с мессенджером
- Асинхронное исполнение, множественное исполнение

Архитектура:

- Модульное RESTful приложение: app/features/ (users, teams, rooms и др.)
- DB-доступ: Через зависимости (get_db)
- Настройки: YAML-конфиг

Используемые технологии / подробности проекта в фронтенд части проекта

Стек технологий:

- Фронтенд: React + Vite
- Стили: Tailwind CSS + shadcn/ui
- Состояние: Jotai + TanStack Query
- Навигация: React Router

Архитектура:

- Модульная структура с изолированными разделами
- Оптимизированные запросы с автоматическим кэшированием
- Динамическая загрузка данных по требованию с помощью atom Jotai

Итерация 1 (Базовый каркас)

Задачи:

- Развернуть бэкенд на FastAPI
- Интегрировать MongoDB и Rocket.Chat API
- Подготовить Docker-окружение
- Создать систему кодогенерации для API

Результаты:

- Рабочее API для взаимодействия с Rocket.Chat
- Настроенная база данных для хранения конфигураций
- Docker-образы для бэкенда
- Сгенерированные клиентские модели для будущего фронтенда

Итерация 2 (Пользователи)

Задачи:

- Вывести таблицу пользователей с данными из Rocket.Chat
- Реализовать клиентскую сортировку и пагинацию
- Добавить UI фильтров (фронтенд)

Результаты:

- Таблица пользователей с актуальными данными
- Возможность сортировки и постраничного просмотра
- Визуальные фильтры (без бэкенд-логики)

Итерация 3 (Групповые операции)

Задачи:

- Реализовать экспорт/импорт CSV
- Настроить SMTP-рассылки
- Добавить страницы сущностей с поиском

Результаты:

- Работа с CSV: загрузка/выгрузка пользователей
- Отправка писем через SMTP
- Детальные страницы пользователей/комнат с поиском

Итерация 4 (Оптимизация и фиксы)

Задачи:

- Настроить связи между сущностями (пользователи-комнаты-команды)
- Исправить критические баги (Docker, токены)
- Добавить документацию

Результаты:

- Полный функционал управления связями
- Стабильная версия приложения
- Оптимизированные Docker-образы
- Инструкции по установке и настройке

Итоги

1. Реализация

Полностью выполнено ТЗ: веб-приложение для автоматизации администрирования Rocket.Chat с управлением пользователями, комнатами и SMTP-рассылкой.

2. Тестирование

Из-за лицензионных ограничений проведено ручное тестирование. Все функции проверены в рабочих условиях.

3. Документация

Готово описание процесса установки и запуска релизной версии.

4. Результат

Решение сокращает время администрирования и готово к промышленному использованию.